Building an Interactive Storymap using {mapgl}

A Visual Journey Through My Life.

Pukar Bhandari

pukar.bhandari@outlook.com

2025-09-03

Table of contents

1 Introduction

Interactive storymaps have become an increasingly popular tool for data visualization and storytelling, particularly in fields like urban planning, transportation analysis, and geographic data science. They allow us to combine spatial data with narrative elements, creating compelling visual stories that engage audiences while conveying complex information effectively.

In this post, I'll walk you through creating an interactive storymap using the {mapgl} R package —a powerful tool that leverages MapLibre GL JS to create stunning, performant web maps. This particular storymap traces my personal journey across different locations where I've lived, studied, and worked, from Nepal to the United States.

2 Why {mapgl} for Storymaps?

The {mapgl} package offers several advantages for transportation planners and data analysts:

- **Performance**: Built on MapLibre GL JS, it handles large datasets and complex visualizations smoothly
- **Interactivity**: Native support for user interactions, animations, and dynamic content
- **Customization**: Extensive styling options and the ability to use custom map tiles
- **Integration**: Seamless integration with Shiny for reactive web applications
- **Open Source**: No API keys required when using open map styles

3 Interactive Storymap Demo

Below is the complete interactive storymap. You can scroll through the different sections to see how the map smoothly transitions between locations, each representing a significant milestone in my journey.

```
#| '!! shinylive warning !!': |
#|   shinylive does not work in self-contained HTML documents.
#|   Please set `embed-resources: false` in your metadata.
#| standalone: true
#| viewerHeight: 600

source("app.R")
```

4 Understanding the Code Structure

The storymap application follows a typical Shiny structure with some specialized {mapgl} components. Following is the source code for this dashboard (Please click on the numbers on the far right for additional explanation of syntax and mechanics):

```
library(shiny)
library(mapgl)

ui <- shiny::fluidPage(
  tags$link(
    href = "https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;600&
display=swap",
    rel="stylesheet"
  ),
  mapgl::story_map(
    map_id = "map",
    font_family = "Poppins",
    sections = list(
      "intro" = mapgl::story_section(
        title = "Introduction",
        content = "Hello I am Pukar Bhandari; and this is my life journey."
      ),
      "birth_place" = mapgl::story_section(
        title = "Birth Place",
        content = "This is where I was born."
```

```
      ),
      "lower_primary" = mapgl::story_section(
        title = "Kankai Education Foundation",
        content = "This is where I studied Nursery to Grade 3."
      ),
      "upper_primary" = mapgl::story_section(
        title = "Laligurans English School",
        content = "This is where I studied Grade 4 & Grade 5."
      ),
      "middle_school" = mapgl::story_section(
        title = "Birat Jyoti English Secondary School",
        content = "This is where I studied Grade 6 & Grade 7."
      ),
      "secondary_school" = mapgl::story_section(
        title = "Little Flowers' English School",
        content = "This is where I studied Grade 8 to Grade 10."
      ),
      "higher_secondary" = mapgl::story_section(
        title = "Kanchanjunga English Higher Secondary School",
        content = "This is where I studied Higher Secondary (Plus 2)."
      ),
      "bachelors" = mapgl::story_section(
        title = "Institute of Engineering Pulchowk Campus",
        content = "This is where I obtained my Bachelor's Degree in
Architecture."
      ),
      "masters" = mapgl::story_section(
        title = "University of Utah",
        content = "This is where I obtained my Master of City & Metropolitan
Planning degree."
      ),
      "work_1" = mapgl::story_section(
        title = "Metro Analytics",
        content = "This is where I lived and worked between 2023-2025."
      ),
      "work_2" = mapgl::story_section(
        title = "Wasatch Front Regional Council",
        content = "This is where I live and work currently."
      )
    ),
    map_type = "maplibre"
  )
)

server <- function(input, output, session) {
  output$map <- mapgl::renderMaplibre({
    mapgl::maplibre(
      style = "https://tiles.openfreemap.org/styles/liberty",
```

```
    center = c(0, 0),
    zoom = 2.75,
    scrollZoom = FALSE
  ) |>
    mapgl::set_projection(projection = "globe") |>
    mapgl::add_globe_control() |>
    mapgl::add_navigation_control(visualize_pitch = TRUE) |>
    mapgl::add_globe_minimap(position = "bottom-left")
})

mapgl::on_section("map", "intro", {
  mapgl::maplibre_proxy("map") |>
    mapgl::clear_markers() |>
    mapgl::fly_to(
      center = c(0, 0),
      zoom = 2.75,
      pitch = 0,
      bearing = 0
    )
})

mapgl::on_section("map", "birth_place", {
  mapgl::maplibre_proxy("map") |>
    mapgl::fly_to(center = c(87.99371750247397, 26.646533355308083),
           zoom = 16,
           pitch = 49,
           bearing = 12.8)
})

mapgl::on_section("map", "lower_primary", {
  mapgl::maplibre_proxy("map") |>
    mapgl::fly_to(center = c(87.99734666704784, 26.646179601882675),
           zoom = 16,
           pitch = 49,
           bearing = 12.8)
})

mapgl::on_section("map", "upper_primary", {
  mapgl::maplibre_proxy("map") |>
    mapgl::fly_to(center = c(88.01116125198942, 26.608294715049524),
           zoom = 16,
           pitch = 49,
           bearing = 12.8)
})

mapgl::on_section("map", "middle_school", {
  mapgl::maplibre_proxy("map") |>
    mapgl::fly_to(center = c(87.99218594227516, 26.620735015314324),
```

```
            zoom = 16,
            pitch = 49,
            bearing = 12.8)
})

mapgl::on_section("map", "secondary_school", {
  mapgl::maplibre_proxy("map") |>
    mapgl::fly_to(center = c(87.99006631463702, 26.63185913816036),
            zoom = 16,
            pitch = 49,
            bearing = 12.8)
})

mapgl::on_section("map", "higher_secondary", {
  mapgl::maplibre_proxy("map") |>
    mapgl::fly_to(center = c(87.99921328178023, 26.633744364809765),
            zoom = 16,
            pitch = 49,
            bearing = 12.8)
})

mapgl::on_section("map", "bachelors", {
  mapgl::maplibre_proxy("map") |>
    mapgl::fly_to(center = c(85.31840215760691, 27.681136558618093),
            zoom = 16,
            pitch = 49,
            bearing = 12.8)
})

mapgl::on_section("map", "masters", {
  mapgl::maplibre_proxy("map") |>
    mapgl::fly_to(center = c(-111.84448004883544, 40.76106105996334),
            zoom = 16,
            pitch = 49,
            bearing = 12.8)
})

mapgl::on_section("map", "work_1", {
  mapgl::maplibre_proxy("map") |>
    mapgl::fly_to(center = c(-84.3938999520061, 33.76728402587881),
            zoom = 16,
            pitch = 49,
            bearing = 12.8)
})

mapgl::on_section("map", "work_2", {
  mapgl::maplibre_proxy("map") |>
    mapgl::fly_to(center = c(-111.90422445885304, 40.76973849954712),
```

```
              zoom = 16,
              pitch = 49,
              bearing = 12.8)
  })
}

shiny::shinyApp(ui, server)
```

Line 1

Load {shiny} package for building reactive web applications

Line 2

Load {mapgl} package for interactive mapping and storymap functionality

Line 4

Define the user interface - what users see and interact with

Line 9

Create the main storymap interface with sidebar and map coordination

Line 12

Define each story section with title and descriptive content

Line 62

Define the server function - handles user interactions and updates

Line 63

Render the interactive MapLibre map with styling and controls

Line 65

Use free OpenFreeMap tiles (no API key required)

Line 70

Set up 3D globe projection for better geographic context

Line 76

Handle section transitions - when user reaches each story step

Line 168

Launch the Shiny application combining UI and server components

Setting Up the User Interface

```
library(shiny)
library(mapgl)

ui <- shiny::fluidPage(
  tags$link(
```

```
    href = "https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;600&
display=swap",
    rel="stylesheet"
  ),
  mapgl::story_map(
    map_id = "map",
    font_family = "Poppins",
    sections = list(
      "intro" = mapgl::story_section(
        title = "Introduction",
        content = "Hello I am Pukar Bhandari; and this is my life journey."
      ),
      "birth_place" = mapgl::story_section(
        title = "Birth Place",
        content = "This is where I was born."
      ),
      # Additional sections...
    ),
    map_type = "maplibre"
  )
)
```

Line 1

Shiny Framework: Essential for creating reactive web applications

Line 2

MapGL Package: Provides the storymap functionality and MapLibre integration

Line 5

Custom Fonts: Loading Google Fonts for better typography (important for professional presentations)

Line 9

Story Map Container: The main wrapper that coordinates the sidebar content with map interactions

Line 11

Typography: Consistent font family improves readability and professional appearance

Line 12

Section Definitions: Each section corresponds to a step in the story with associated map behaviors

Line 23

Map Engine: Using MapLibre GL JS for performant vector tile rendering

Configuring the Map

```
server <- function(input, output, session) {
  output$map <- mapgl::renderMaplibre({
    mapgl::maplibre(
      style = "https://tiles.openfreemap.org/styles/liberty",
      center = c(0, 0),
      zoom = 2.75,
      scrollZoom = FALSE
    ) |>
      mapgl::set_projection(projection = "globe") |>
      mapgl::add_globe_control() |>
      mapgl::add_navigation_control(visualize_pitch = TRUE) |>
      mapgl::add_globe_minimap(position = "bottom-left")
  })
}
```

Line 2

Render Function: Creates the reactive map output that responds to user interactions

Line 4

Open Map Style: Using OpenFreeMap's Liberty style (free, no API key required)

Line 5

Initial Center: Starting at global view (longitude 0, latitude 0)

Line 6

Zoom Level: Appropriate for showing global context initially

Line 7

Scroll Control: Disabled to prevent interference with story navigation

Line 9

Globe Projection: Creates a 3D globe effect for better geographical context

Line 10

Globe Controls: Adds rotation controls for the 3D globe

Line 11

Navigation Tools: Zoom and rotation controls with pitch visualization

Line 12

Minimap: Provides geographic context when zoomed in on specific locations

Implementing Section Behaviors

```
# Example section handler
mapgl::on_section("map", "birth_place", {
  mapgl::maplibre_proxy("map") |>
    mapgl::fly_to(
```

```
      center = c(87.99371750247397, 26.646533355308083),
      zoom = 16,
      pitch = 49,
      bearing = 12.8
    )
})
```

Line 2

Section Trigger: Responds when user navigates to the “birth_place” section

Line 3

Map Proxy: Allows server-side control of the existing map instance

Line 4

Smooth Transition: `fly_to()` creates animated transitions between locations

Line 5

Coordinates: Precise longitude/latitude coordinates for the specific location

Line 6

Detail Zoom: High zoom level to show local context and landmarks

Line 7

Viewing Angle: Tilted perspective provides better spatial understanding

Line 8

Map Rotation: Oriented to show the location from the most informative angle

5 Best Practices for Transportation Data Storymaps

Based on my experience as a transportation planner, here are key considerations when creating storymaps for professional use:

Data Preparation

- **Coordinate Accuracy**: Verify coordinates using reliable sources (Google Maps, OpenStreetMap)
- **Projection Considerations**: Understand how different projections affect your data representation
- **Performance Optimization**: For large datasets, consider data aggregation and level-of-detail strategies

Visual Design

- **Consistent Styling**: Maintain uniform zoom levels, pitch, and bearing for similar content types
- **Smooth Transitions**: Use appropriate transition speeds (typically 1-3 seconds)
- **Color Schemes**: Choose colors that work well with your base map style
- **Typography**: Select readable fonts that complement your organization’s branding

User Experience

- **Logical Flow**: Organize sections in a meaningful sequence (chronological, thematic, etc.)
- **Clear Navigation**: Provide obvious cues for how users should interact with the storymap
- **Performance**: Test on different devices and internet speeds
- **Accessibility**: Ensure content is accessible to users with disabilities

6 Loading External App Files

To load the complete application from an external `app.R` file (which is cleaner for larger applications), you can modify the {shinylive} block:

````
#| eval: false

# In your index.qmd file
```{shinylive-r}
#| standalone: true
#| viewerHeight: 600
#| components: [editor, viewer]

# Load the complete application
source("app.R")
```
````

This approach allows you to: - Maintain cleaner separation between documentation and code - Enable collaborative development on the application - Version control your application code separately - Reuse the application in different contexts

7 Conclusion

The {mapgl} package provides a powerful platform for creating engaging, interactive storymaps that can effectively communicate spatial narratives. Whether you're documenting personal journeys, presenting planning scenarios, or analyzing transportation patterns, this approach offers the flexibility and performance needed for professional applications.

The combination of R's data processing capabilities with modern web mapping technology creates opportunities for innovative visualization approaches that can enhance public engagement and decision-making processes in transportation planning and urban analytics.

8 Additional Resources

- {mapgl} package documentation
- MapLibre GL JS documentation
- OpenFreeMap styles
- Quarto documentation

This post demonstrates practical applications of open-source geospatial tools for transportation planning and data visualization. All code is available for adaptation to your own projects and use cases.